Project 4

CS 332, Fall 2025

Ben Cole & Koshi Harashima

December 4, 2025

Outline

1. Part 1: Online Reserve Pricing
2. Part 2: Online Revenue Maximization

Outline

1 Part 1: Online Reserve Pricing

2 Part 2: Online Revenue Maximization

Setting

- Repeated single-item second-price auction (SPA) with reserve r.
- Each round: m bidders with i.i.d. values from distribution F; truthful bidding is dominant.
- Round revenue:

$$\mathrm{rev} = \begin{cases} 0, & \max_i v_i < r \\ \max\{r, \text{2nd-highest}(v)\}, & \text{otherwise.} \end{cases}$$

- **Goal**: learn a revenue-maximizing reserve r online.
- **Evaluation**: average revenue vs. time; comparison to the optimal fixed reserve for F.

- **Bandit baseline (epsilon-greedy)** over a discretized reserve grid $\mathcal{G} \subset [0,1]$.
  Exploration $\varepsilon_t = \min\{1, c/\sqrt{t}\}$; otherwise exploit the best empirical arm.
- **Myerson plug-in (empirical CDF)**:
  maintain empirical CDF $\hat{F}$ of observed bids and choose

$$r_t = \arg\max_{z \in \mathcal{G}} z \cdot \big(1 - \hat{F}(z)\big).$$

Since the mechanism is a second-price auction, truthful bidding is a dominant strategy; hence the seller observes bidders' valuations, providing full-information feedback for learning.

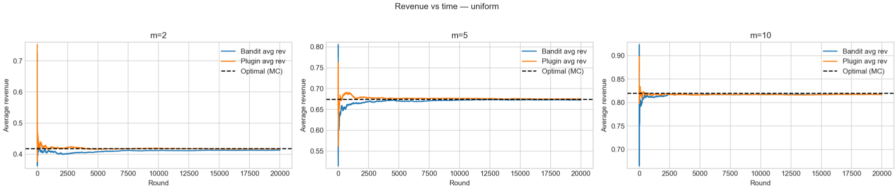
Simulation Setup

- **Distributions** F: Uniform$[0, 1]$; quadratic $F(z) = z^2$ on $[0, 1]$ via inverse CDF $z = \sqrt{q}$.
- **Bidders**: $m \in \{2, 5, 10\}$.
- **Horizon**: $T = 20{,}000$ rounds.
- **Grids**: bandit $|\mathcal{G}| = 101$; plug-in $|\mathcal{G}| = 1001$.
- **Metrics**: average revenue vs. time; chosen reserve trajectories; MC optimal baseline.

Benchmarks

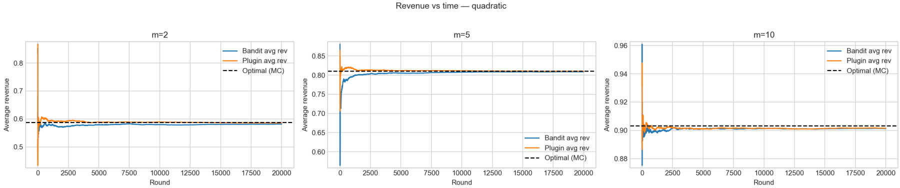
- **Optimal fixed reserve (Monte Carlo)**: for each (F, m), grid search over r with many rounds.
- Analytic check for $F = \mathrm{Uniform}[0, 1]$, $m = 2$: $r^\star = \frac{1}{2}$ and $\mathbb{E}[\mathrm{rev}^\star] = \frac{5}{12} \approx 0.4167$.

Results (Uniform): Revenue vs. time



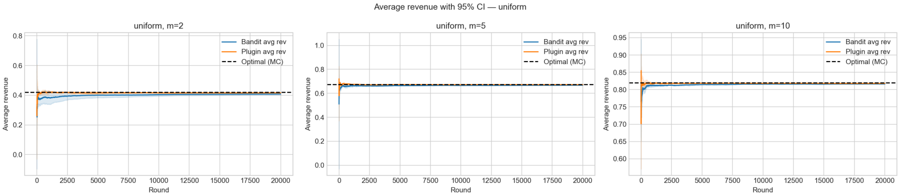
Bandit and plug-in converge to the MC optimal revenue; plug-in typically locks on faster.

Results (Quadratic $F(z) = z^2$): Revenue vs. time

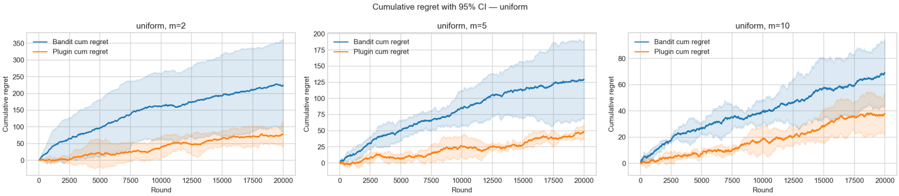


Qualitatively similar convergence; the optimal reserve shifts due to a different F.

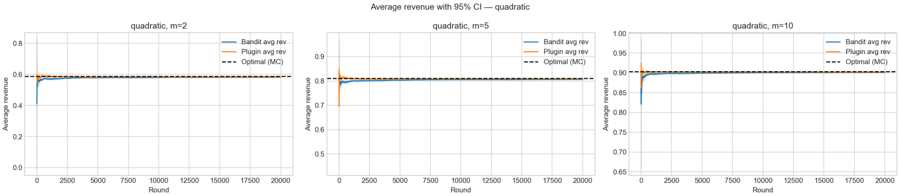
Average revenue with 95% CI (Uniform)



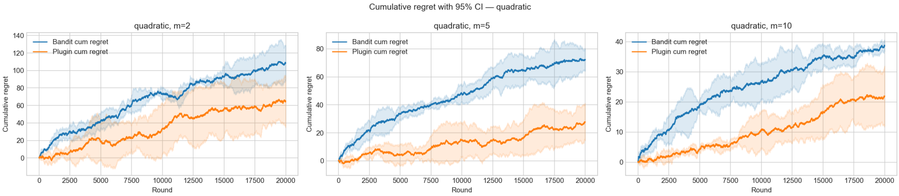
Cumulative regret with 95% CI (Uniform)



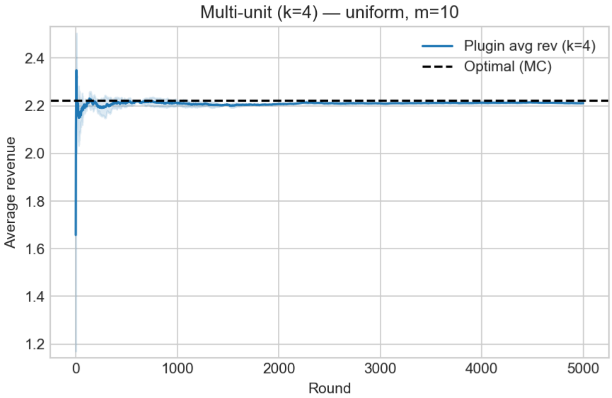
Average revenue with 95% CI (Quadratic)



Cumulative regret with 95% CI (Quadratic)



Multi-unit extension (example: $k = 4$ items)



Reserve-learning via the plug-in extends naturally; pricing uses $(k+1)$-st bid with reserve floor. The clearing price equals the 5th-highest valuation, i.e., the 6th order statistic out of 11. Hence its expectation is $\mathbb{E}[V_{(6:11)}] = \frac{6}{11}$ Since 4 units are sold at this uniform price, the seller's expected revenue is $\frac{24}{11} \approx 2.18$.

Part 1: Key Takeaways

- Both methods approach the optimal fixed-reserve revenue across tested (F, m)(without heterogeneity).
- Plug-in converges faster by leveraging structure $r(1 - \hat{F}(r))$.
- **More bidders (m)** increase expected revenue and accelerate convergence.
- **More items (k)** (multi-unit) can be handled with truthful $(k+1)$-st pricing and a reserve; behavior mirrors the single-item case with higher throughput.
- Grid resolution trades precision for compute; plug-in benefits from a finer grid.

Outline

1. Part 1: Online Reserve Pricing
2. Part 2: Online Revenue Maximization

Part 2: Setting

- Positions with click probabilities $w_1 \geq \cdots \geq w_m$; all agents have values $v_i \sim F_i$ (independent).
- **Truthful baselines**: VCG (welfare-optimal) and Myerson (revenue-optimal via virtual values).
- **Online learner**: plug-in Myerson — estimate F_i empirically, compute $\hat{\phi}_i$, allocate to maximize $\sum_j w_j \hat{\phi}_i$.
- We use expected clicks to measure revenue (no Bernoulli noise) for clearer convergence diagnostics.

Part 2: Theory (VCG vs Myerson)

- **VCG**: allocate by descending v_i; payments equal externality to others.
- **Myerson**: compute $\phi_i(v_i)$ and allocate to maximize $\sum_j w_j \phi_i(v_i)$.
  Specifically, bidders with non-positive virtual values are excluded,
  and the remaining bidders are ranked by their virtual values.

  Positions are then assigned in decreasing order of click-through rates to the bidders with the highest virtual values.
- Oracle Myerson uses true F_i; plug-in replaces ϕ_i with $\hat{\phi}_i$ from empirical CDFs.

Part 2: A New Learning Algorithm

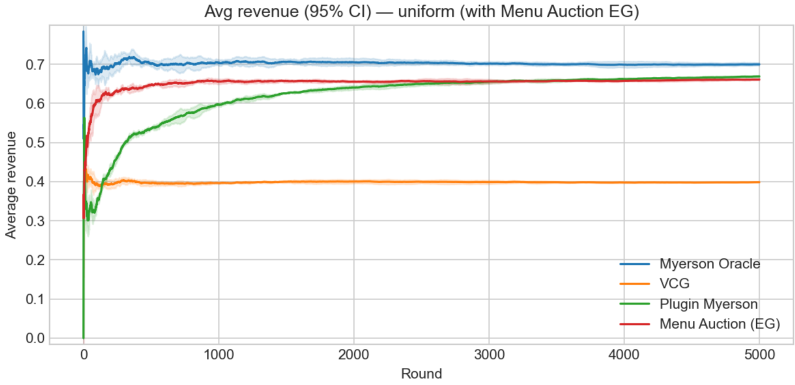
Rather than learning a single reserve price, we design a *menu auction* that posts a price for each slot. The mechanism presents bidders with a menu of $(\text{slot}, \text{price})$ pairs, and bidders choose sequentially according to their utilities(to avoid the discussion of feasibilty).

- Truthful: choosing the utility-maximizing option is a dominant strategy.
- Robust to heterogeneous bidders
- Learnable: the posted prices can be updated online using learning algorithms.

Part2:Comparison

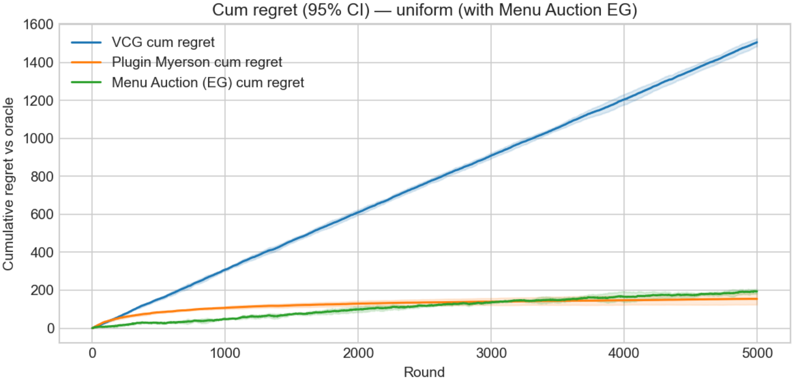
	VCG	**Empirical Myerson**	**Menu Pricing**
Convergence target	const. to optimal	optimal	suboptimal
Convergence speed	–	fast	relatively slow
Truthfulness (static)	Yes	Yes	Yes
Truthfulness (dynamic)	Yes	No	No
Robustness to manipulation	High	Low	Medium
Heterogeneity robustness	High	Medium	High (price discr.)
Online learning	No (static)	Yes (static)	Yes

Part 2: Results (example)



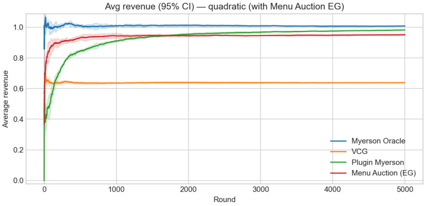
Average revenue vs. time: oracle Myerson (reference), VCG (welfare), plug-in (learned), and Menu Posting

Part 2: Regret (Uniform)

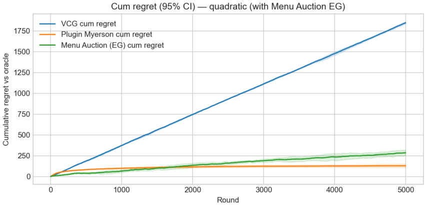


Regret

Part 2: 95% CI and Regret (Quadratic)

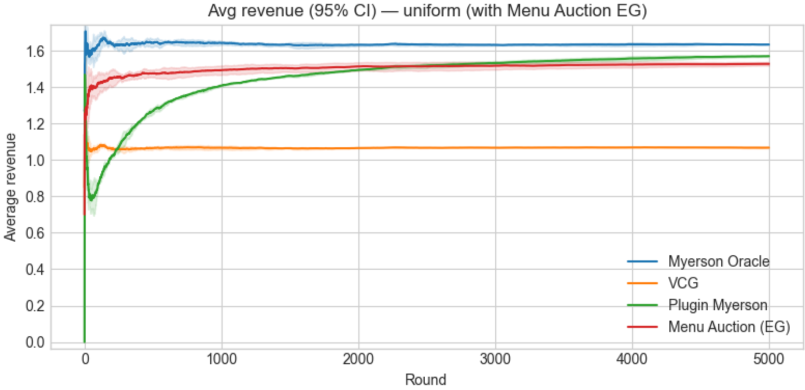


95% CI of Revenue



Regret

Part 2: Sweeps over m and w

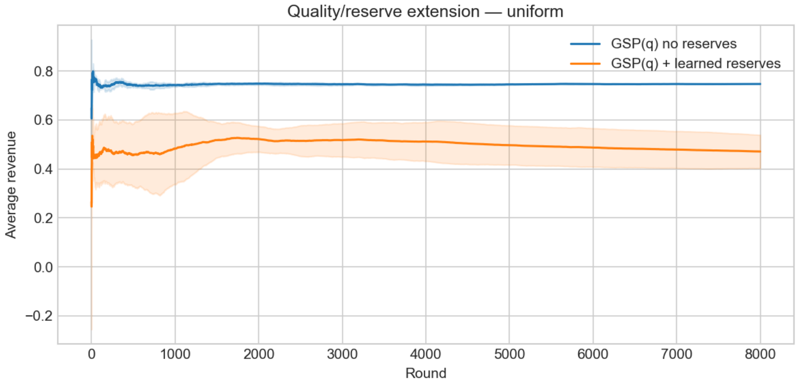


More bidders/stronger top slots $\Rightarrow$ higher revenue and faster stabilization.

Extension: Quality scores and slot reserves

- **Quality scores** q_i: rank by effective score $q_i v_i$ (GSP-style baseline for context).
- **Slot reserves** $r_j(t)$: per-slot thresholds learned online (simple bandit over reserve grid).
- Compare revenue/allocation with and without quality/reserves; reserves stabilize; revenue can improve.

Extension: Example outcome



With quality scores, learned slot reserves raise revenue relative to no-reserve baseline (context-dependent).

Part 2: Takeaways

- **Mech. design**: VCG (welfare) and Myerson (revenue) provide truthful references; plug-in learns Myerson, and Menu auction learns suboptimal menu.
- **Learning**: Plug-in converges to oracle revenues; CI bands shrink; regret grows sublinearly.
- **Comparatives**: Higher m / stronger w lift revenue and speed stabilization.
- **Extension**: Quality scores change ranking; per-slot reserves can lift revenue; reserves stabilize.

AI & Collaboration

AI was used for coding assistance, organization, and plotting utilities. All results were generated from our own code; no external algorithm implementations were imported.